Proyecto Final Micro

EUSART (ENHANCED UNIVERSAL SYNCHRONOUS ASYNCHRONOUS RECEIVER TRANSMITTER)

El EUSART es un módulo de hardware que permite enviar y recibir datos seriales sin que la CPU tenga que manejar bit por bit la comunicación. Internamente incluye:

- Generador de baud rate
- Registro de transmisión
- Registro de recepción
- Detección de errores
- Comunicación síncrona y asíncrona

FIGURE 12-1: EUSART TRANSMIT BLOCK DIAGRAM

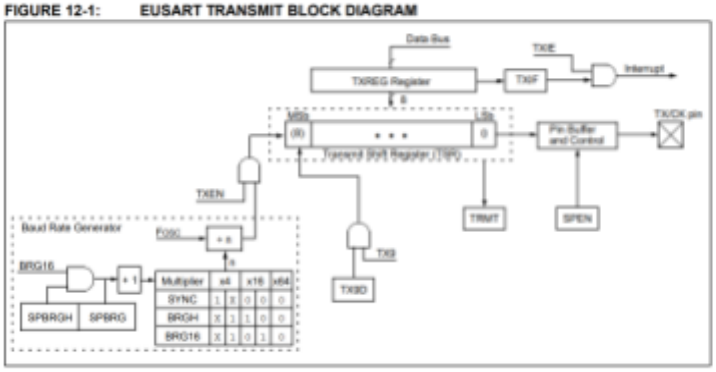
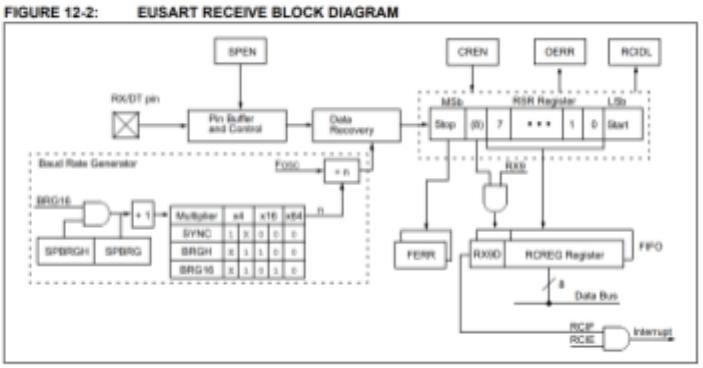


FIGURE 12-2: EUSART RECEIVE BLOCK DIAGRAM



REGISTROS UTILIZADOS:

1. TXSTA
2. RCSTA
3. TXREG
4. SPBRG
5. PIR

TXSTA

REGISTER 12-1: TXSTA: TRANSMIT STATUS AND CONTROL REGISTER

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R-1	R/W-0
CSRC	TX9	TXEN(1)	SYNC	SENDB	BRGH	TRMT	TX9D
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7 **CSRC:** Clock Source Select bit
Asynchronous mode:
Don't care
Synchronous mode:
1 = Master mode (clock generated internally from BRG)
0 = Slave mode (clock from external source)

bit 6 **TX9:** 9-bit Transmit Enable bit
1 = Selects 9-bit transmission
0 = Selects 8-bit transmission

bit 5 **TXEN:** Transmit Enable bit(1)
1 = Transmit enabled
0 = Transmit disabled

bit 4 **SYNC:** EUSART Mode Select bit
1 = Synchronous mode
0 = Asynchronous mode

bit 3 **SENDB:** Send Break Character bit
Asynchronous mode:
1 = Send Sync Break on next transmission (cleared by hardware upon completion)
0 = Sync Break transmission completed
Synchronous mode:
Don't care

bit 2 **BRGH:** High Baud Rate Select bit
Asynchronous mode:
1 = High speed
0 = Low speed
Synchronous mode:
Unused in this mode

bit 1 **TRMT:** Transmit Shift Register Status bit
1 = TSR empty
0 = TSR full

bit 0 **TX9D:** Ninth bit of Transmit Data
Can be address/data bit or a parity bit.

Note 1: SREN/CREN overrides TXEN in Sync mode.

Viendo esto, utilizaremos este periférico como un UART normal, por lo que lo pondremos en modo asíncrono. El bit que habilita la transmisión es TXEN, por lo que se pone el bit 5 en "1". Y para una resolución más fina en la generación de Baude Rate (más adelante se explicará esto), utilizamos "High Speed", entonces se configuraría de esta manera:

TXSTA = 0b00100100;
En hexadecimal sería: TXSTA = 0x24;

Registro TXREG y otros bits importantes de transmisión

TXREG (Transmit Register): Es un registro del PIC donde se guarda el dato que se quiera mandar. Este registro es de 1 byte, por lo que solo se pueden almacenar 1 palabra binaria de hasta 8 bits. Por eso se suelen solo mandar caracteres cuyo código ASCII entre en la capacidad de almacenamiento, como ‘A’ o ‘H’.

TRMT: Este otro bit del TXSTA es importante y se usa en el código. Este bit funciona como una bandera del estado del TSR (Transmit Shift Register), que es otro registro interno del PIC, que toma el mensaje que quieres enviar, y agrega los start y stop bit, y después los va enviando uno por uno. Por lo que TSR es un intermediario:

TXREG (registro donde escribes el mensaje) → TSR (registro intermediario) → Pin TX (salida)

Ejemplo: Si quieres mandar “A”, en MPLAB harías:

```
TXREG = 'A';
```

El código ASCII de “A” es 01000001

Lo que sucede internamente en el PIC sería:

```
TXREG = 01000001
TSR   = vacío
```

Después, cuando se quiere mandar el dato, ocurre:

```
TXREG = vacío
TSR   = 01000001
```

TXREG queda libre, y se puede volver a cargar con otro dato

TSR desplaza los bits así:

Start	D0	D1	D2	D3	D4	D5	D6	D7	Stop
0	1	0	0	0	0	0	1	0	1

Y los envía uno por uno a la velocidad configurada por el Baude Rate

Aquí es cuando entra TMRT. Este bit funciona como estado de lo que sucede con TSR. Mientras TSR configura la palabra con los start y stop bits, y los va mandando, TRMT = 0, porque TSR está ocupado con el proceso de desplazamiento del dato. Una vez termina:

```
TSR = vacío
TRMT = 1
```

Significando que se desplazó el dato, y está listo para volver a desplazar otro dato.

SPBRG (Serial Port Baud Rate Generator Register)

Es un registro donde se guarda un número que utiliza el generador de baud rate para producir la frecuencia de transmisión deseada.

Fosc (reloj del PIC) → Generador de Baud Rate → SPBRG → 9600, 19200 bps, etc.

De acuerdo al Datasheet, la fórmula empleada para calcular dicha frecuencia depende de cómo se configure.

TABLE 12-3: BAUD RATE FORMULAS

Configuration Bits			BRG/EUSART Mode	Baud Rate Formula
SYNC	BRG16	BRGH		
0	0	0	8-bit/Asynchronous	Fosc/[64 (n+1)]
0	0	1	8-bit/Asynchronous	Fosc/[16 (n+1)]
0	1	0	16-bit/Asynchronous	
0	1	1	16-bit/Asynchronous	Fosc/[4 (n+1)]
1	0	x	8-bit/Synchronous	
1	1	x	16-bit/Synchronous	

Legend: x = don't care, n = value of SPBRGH, SPBRG register pair

De la manera en la que lo estamos configurando (001) sería:

$$BR = \frac{F_{OSC}}{16\,(SPBRG+1)}$$

o

$$SPBRG = \frac{F_{OSC}}{16\,(BR)} - 1$$

Si lo configuramos para un Baude Rate de 9600 bps, el SPBRG sería

$$SPBRG = \frac{8{,}000{,}000\ Hz}{16\,(9{,}600)} - 1 \approx 51.08$$

Que se redondeará a 51 con esto, obtendremos entonces un Baude Rate REAL de:

$$BR = \frac{8{,}000{,}000\ Hz}{16\,(51+1)} \approx 9{,}615.38\ bps$$

Muy cercano al valor deseado de 9600, con un error porcentual de:

$$Error\ (\%) = \frac{BR_{Real} - BR_{Deseado}}{BR_{Deseado}}(100) = \frac{9615.38 - 9600}{9600}(100) \approx 0.1602\ \%$$

RCSTA

REGISTER 12-2: RCSTA: RECEIVE STATUS AND CONTROL REGISTER(1)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R-0	R-0	R-x
SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7 **SPEN:** Serial Port Enable bit
1 = Serial port enabled (configures RX/DT and TX/CK pins as serial port pins)
0 = Serial port disabled (held in Reset)

bit 6 **RX9:** 9-bit Receive Enable bit
1 = Selects 9-bit reception
0 = Selects 8-bit reception

bit 5 **SREN:** Single Receive Enable bit
Asynchronous mode:
Don't care
Synchronous mode – Master:
1 = Enables single receive
0 = Disables single receive
This bit is cleared after reception is complete.
Synchronous mode – Slave
Don't care

bit 4 **CREN:** Continuous Receive Enable bit
Asynchronous mode:
1 = Enables receiver
0 = Disables receiver
Synchronous mode:
1 = Enables continuous receive until enable bit CREN is cleared (CREN overrides SREN)
0 = Disables continuous receive

bit 3 **ADDEN:** Address Detect Enable bit
Asynchronous mode 9-bit (RX9 = 1):
1 = Enables address detection, enable interrupt and load the receive buffer when RSR<8> is set
0 = Disables address detection, all bytes are received and ninth bit can be used as parity bit
Asynchronous mode 8-bit (RX9 = 0):
Don't care

bit 2 **FERR:** Framing Error bit
1 = Framing error (can be updated by reading RCREG register and receive next valid byte)
0 = No framing error

bit 1 **OERR:** Overrun Error bit
1 = Overrun error (can be cleared by clearing bit CREN)
0 = No overrun error

bit 0 **RX9D:** Ninth bit of Received Data
This can be address/data bit or a parity bit and must be calculated by user firmware.

De este registro, solo interesa (hasta el momento en que escribo esto xd), el bit 7 (SPEN) y el bit 4 (CREN). SPEN configura los pines RX/DT y TX/CK (RC6 y RC7) como puertos seriales, por lo que lo necesitamos activar. Y CREN habilita el receptor. Entonces, lo configuramos de esta manera:

RCSTA = 0b10010000;
En hexadecimal sería: RCSTA = 0x90;

Otros bits importantes del RCSTA

OERR: Este bit del registro RCSTA indica que el buffer de recepción del EUSART se llenó porque el programa no leyó los datos recibidos a tiempo. Cuando este error ocurre, el receptor deja de aceptar nuevos datos. Para eliminarlo es necesario limpiar el bit CREN, es decir, deshabilitar y volver a habilitar la recepción continua mediante el bit CREN.

PIR1 (PERIPHERAL INTERRUPT REQUEST REGISTER 1)

Este registro almacena en sus bits las banderas de interrupción de varios periféricos, como los del ADC, TIMER1, TIMER2, el CCP1, y por supuesto, las banderas de transmisión y recepción TXIF y RXIF del periférico de EUSART del PIC16F887.

REGISTER 2-6: PIR1: PERIPHERAL INTERRUPT REQUEST REGISTER 1

U-0	R/W-0	R-0	R-0	R/W-0	R/W-0	R/W-0	R/W-0
—	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7 **Unimplemented:** Read as '0'

bit 6 **ADIF:** A/D Converter Interrupt Flag bit
1 = A/D conversion complete (must be cleared in software)
0 = A/D conversion has not completed or has not been started

bit 5 **RCIF:** EUSART Receive Interrupt Flag bit
1 = The EUSART receive buffer is full (cleared by reading RCREG)
0 = The EUSART receive buffer is not full

bit 4 **TXIF:** EUSART Transmit Interrupt Flag bit
1 = The EUSART transmit buffer is empty (cleared by writing to TXREG)
0 = The EUSART transmit buffer is full

bit 3 **SSPIF:** Master Synchronous Serial Port (MSSP) Interrupt Flag bit
1 = The MSSP interrupt condition has occurred, and must be cleared in software before returning from the Interrupt Service Routine. The conditions that will set this bit are:
SPI
A transmission/reception has taken place
I^2C Slave/Master
A transmission/reception has taken place
I^2C Master
The initiated Start condition was completed by the MSSP module
The initiated Stop condition was completed by the MSSP module
The initiated restart condition was completed by the MSSP module
The initiated Acknowledge condition was completed by the MSSP module
A Start condition occurred while the MSSP module was idle (Multi-master system)
A Stop condition occurred while the MSSP module was idle (Multi-master system)
0 = No MSSP interrupt condition has occurred

bit 2 **CCP1IF:** CCP1 Interrupt Flag bit
Capture mode:
1 = A TMR1 register capture occurred (must be cleared in software)
0 = No TMR1 register capture occurred
Compare mode:
1 = A TMR1 register compare match occurred (must be cleared in software)
0 = No TMR1 register compare match occurred
PWM mode:
Unused in this mode

bit 1 **TMR2IF:** Timer2 to PR2 Interrupt Flag bit
1 = A Timer2 to PR2 match occurred (must be cleared in software)
0 = No Timer2 to PR2 match occurred

bit 0 **TMR1IF:** Timer1 Overflow Interrupt Flag bit
1 = The TMR1 register overflowed (must be cleared in software)
0 = The TMR1 register did not overflow

De todas las banderas que se almacena este registro, la que interesa es RXIF. Cuando RXIF=0, significa que aún no se ha recibido ningún dato. Cuando RXIF=1, la bandera indica que se ha recibido un nuevo dato. Esta bandera es utilizada en la función *bool UART_Data_Ready(void)* del protocolo de comunicación del código del proyecto final. En esta función, se aprovecha de dicha bandera, para en cada ciclo del *while(1),* estar revisando si es que se ha recibido un dato o no. En el caso de que RXIF=0, el programa corre regularmente. Cuando RXIF=1, significa que ha llegado un nuevo mensaje/dato, y que se está listo para leerlo y proceder con lo designado por el programa principal.

Referencias:

Microchip Technology Incorporated. (2007). *PIC16F882/883/884/886/887 data sheet: 28/40/44-pin enhanced flash-based 8-bit CMOS microcontrollers* (DS41291D). Microchip Technology Incorporated.